

JAVASCRIPT MODERNE

JS : Communiquer avec une API (fetch, async/await)

Sommaire

-  1. Qu'est-ce qu'une API ?
Comprendre l'échange de données.
-  2. Le problème : L'asynchronisme
Pourquoi le navigateur ne doit pas se figer.
-  3. La syntaxe moderne : async / await
Le standard de l'écriture asynchrone.
-  4. Les 3 étapes d'un appel standard
Le schéma universel de fetch.
-  5. Sécuriser avec try...catch
Gérer les pannes réseau sans planter.
-  6. Exercice 2 : Pratique guidée
Énoncé et contraintes de l'exercice.
-  7. Correction pas-à-pas
Explications de la structure et du code final.

| 01. Qu'est-ce qu'une API ?

Une API (Application Programming Interface) permet à votre application de communiquer avec un autre serveur pour récupérer ou envoyer des données (météo, utilisateurs, produits, etc.).



02. Le problème : L'asynchronisme

Récupérer des données sur le réseau prend du temps (de quelques millisecondes à plusieurs secondes selon la connexion).

Le risque de blocage

Le navigateur ne doit pas se "figer" ou bloquer l'affichage de la page pendant ce temps d'attente réseau.

Solution : Le code asynchrone

JavaScript utilise donc du code asynchrone :

-  Le script continue de s'exécuter en arrière-plan pendant la requête.
-  Il traite la réponse de l'API uniquement lorsqu'elle arrive.

| 03. La syntaxe moderne : `async` / `await`

C'est la syntaxe standard et actuelle pour gérer l'asynchronisme en JS. Elle permet d'écrire du code asynchrone qui ressemble à du code "normal" (synchrone), ce qui le rend beaucoup plus lisible.



`async`

Se place obligatoirement devant la déclaration d'une fonction pour indiquer qu'elle va effectuer des tâches asynchrones.



`await`

Se place devant l'instruction asynchrone (comme `fetch`). Il dit au code : *"Attends que cette étape soit terminée avant de passer à la suivante"*.



Lisibilité

Évite d'imbriquer des callbacks ou d'enchaîner trop de blocs `.then()` difficiles à relire.

04. Les 3 étapes d'un appel API standard

Pour récupérer et exploiter des données, le schéma est toujours le même :

- 1 Déclarer la fonction :
Ajouter le mot-clé `async` devant.
- 2 Appeler & Attendre :
Utiliser `await fetch()` pour obtenir la réponse brute.
- 3 Traduire la réponse :
Convertir le flux brut en objet JSON exploitable.

```
APP.JS

// 1. Déclarer la fonction asynchrone
async function chargerDonnees() {
  // 2. Envoyer la requête et attendre la réponse brute
  const reponse = await fetch("https://api.exemple.com/d
  // 3. Traduire la réponse brute en JSON
  const donnees = await reponse.json();
  console.log(donnees); // Prêtes à l'emploi !
}
```

05. Sécuriser son code avec try...catch

Le réseau n'est pas fiable à 100% (URL mal écrite, serveur en panne, connexion coupée). Sans protection, une erreur réseau arrête complètement votre script.

🛡️ Le bloc try...catch

- ▶ try : On y met le code "à risque" dépendant du réseau.
- ⊗ catch : Intercepte l'erreur proprement en évitant le crash.

À retenir : try...catch ne remplace pas les 3 étapes de fetch, il s'enroule simplement autour pour les sécuriser.

```
APP_SECURE.JS

async function chargerDonnees() {
  try {
    const reponse = await fetch("https://api.com/data")
    const donnees = await reponse.json();
    console.log(donnees);
  } catch (erreur) {
    console.log("Une erreur est survenue :", erreur);
  }
}
```

Exercice 2 : Contacter une API

Mettez en pratique vos connaissances asynchrones pour interroger une API de tâches réelles en ligne et manipuler le DOM.

🎯 Consignes et Objectifs

- Créez deux fichiers locaux : index.html et main.js.
- Connectez-vous à l'API via le EndPoint ci-dessous.
- Récupérez et affichez le titre des 5 premières tâches dans une liste Web.

API ENDPOINT

https://jsonplaceholder.typicode.com/todos?_limit=5

```
INDEX.HTML

<!DOCTYPE html>
<html lang="fr">
<head>
  <title>Ma Liste API</title>
  <script src="main.js" defer></script>
</head>
<body>
  <h1>Mes Tâches Importées</h1>
  <ul id="taskList"></ul>
</body>
</html>
```


Correction : Connexion & Fetch (1/2)

La première partie cible l'interface utilisateur et déclenche la connexion asynchrone sécurisée au serveur.

-  Ciblage DOM : On référence le conteneur via son ID exact "taskList".
-  Fetch asynchrone : fetch() lance la requête et attend la réponse brute du serveur API en arrière-plan.
-  Conversion JSON : On traduit le corps de cette réponse brute pour le transformer en un tableau d'objets JS utilisable.

```
MAIN.JS (PARTIE 1)

// 1. Sélection de la liste HTML par son identifiant (ID)
const taskList = document.getElementById("taskList");

// 2. Déclaration de la fonction asynchrone
async function chargerTaches() {
  try {
    // 3. Envoi de la requête à l'API et attente de la réponse
    const reponse = await fetch(
      "https://jsonplaceholder.typicode.com/todos?_limit=5"
    );

    // 4. Conversion de la réponse brute au format JSON
    const taches = await reponse.json();

    // Suite à la diapositive suivante...
```

Correction : Injection DOM & Erreurs (2/2)

La seconde partie traite les données reçues, les sécurise, génère les éléments HTML et gère d'éventuelles pannes.

- 🔹 Méthode slice : Par sécurité locale, on force l'extraction d'exactement 5 items (slice(0, 5)).
- 🔗 Boucle dynamique : createElement("li") est appelé à chaque tour de boucle pour créer de nouveaux nœuds HTML.
- 🛑 Bloc catch : Empêche le plantage général et avertit l'utilisateur proprement en injectant un message dans la liste.

```
MAIN.JS (PARTIE 2)

// Sécurité : On s'assure de ne prendre que les 5 premiers
const cinqPremieresTaches = taches.slice(0, 5);

// 5. Parcours du tableau de données
cinqPremieresTaches.forEach((tache) => {
  // Création d'un élément de liste
  const li = document.createElement("li");

  // Ajout du titre de la tâche
  li.textContent = tache.title;

  // Insertion du dans le parent
  taskList.appendChild(li);
});

} catch (erreur) {
  // 6. Gestion de l'erreur en cas d'échec
  taskList.innerHTML = "Impossible de charger les tâches.";
  console.error("Erreur API :", erreur);
}

// 7. Appel de la fonction
chargerTaches();
```

| 07. Aide-mémoire / À retenir absolument

 API & JSON : Récupération de données externes au format universel JSON.

 Asynchronisme : JS n'attend pas, il exécute la suite et traite la réponse à son arrivée.

 Le Duo : `async` qualifie la fonction, `await` suspend l'instruction.

 Trilogie : `fetch()` → `.json()` → manipulation.

 Sécurité : Utilisez impérativement `try...catch` autour de `fetch`.

```
async function nomDeLaFonction() {  
  try {  
    const reponse = await fetch("URL_DE_L_API");  
    const donnees = await reponse.json();  
    // ... utiliser les données ici  
  } catch (erreur) {  
    console.log("Erreur :", erreur);  
  }  
}
```